FPT UNIVERSITY | FPTU QUY NHON AI CAMPUS

FU CAHATA • REFERENCE DOCS

OmniCast - Database Design Specification

Version: 2.0
Created: 2026-09-23
Database: PostgreSQL (Supabase)
ORM: Prisma

Table of Contents

1. Database Overview
2. Prisma Schema
3. SQL Migrations
4. Entity Relationship Diagram
5. Sample Data
6. API Endpoints

1. Database Overview

Connection Details

```
Host: db.tktexqtqfnlfynbytjpw.supabase.co
Port: 5432
Database: postgres
User: postgres

Connection String:
postgresql://postgres:[YOUR-PASSWORD]@db.tktexqtqfnlfynbytjpw.supabase.co:5432/postgres

Supabase URL: https://tktexqtqfnlfynbytjpw.supabase.co
```

Module Structure

```
OmniCast Database
├── Module 1: EPG (Electronic Program Guide)
│   ├── BroadcastChannel, BroadcastProgram
│   ├── ProductionTag, BroadcastAiReport
│   └── Watchlist, ScheduleReminder
│
├── Module 2: Live Streaming & VOD
│   ├── LiveChannel, LiveEvent, Recording
│   ├── Content Sources: External | Uploaded | Generated
│   └── Podcast, PodcastEpisode
│
├── Module 3: Social & Engagement
│   ├── Follow, Subscription
│   ├── Comment, CommentLike
│   ├── Reaction, WatchHistory
│   └── Notification
│
└── Module 4: Core
    ├── User, RefreshToken
    └── AuditLog
```

2. Prisma Schema

```
// This file is the source of truth.
// Run: npx prisma generate
// Run: npx prisma db push

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// ═══════════════════════════════════════════════════════════
// ENUMS
// ═══════════════════════════════════════════════════════════

enum UserRole {
  GUEST
  VIEWER
  STAFF
  ADMIN
}

enum BroadcastCategory {
  GENERAL
  NEWS
  ENTERTAINMENT
  SPORTS
  MOVIES
  KIDS
  DOCUMENTARY
  MUSIC
  EDUCATION
  LIFESTYLE
}

enum LiveCategory {
  GAMING
  PODCAST
  MUSIC
  EDUCATION
  LIFESTYLE
  NEWS
  SPORTS
  TECH
  ART
  BUSINESS
  HEALTH
  FOOD
  TRAVEL
  ENTERTAINMENT
}

enum ContentSource {
  EXTERNAL
  UPLOADED
  GENERATED
}

enum ExternalPlatform {
```

```
  YOUTUBE
  VIMEO
  TWITCH
  FACEBOOK
  TIKTOK
  DAILYMOTION
  CUSTOM_HLS
  CUSTOM_RTMP
  CUSTOM_MP4
  EMBED_IFRAME
}

enum EventStatus {
  SCHEDULED
  LIVE
  ENDED
  CANCELLED
  ON_DEMAND
}

enum ContentType {
  VIDEO
  AUDIO
  LIVE
}

enum StreamQuality {
  AUTO
  SD_480P
  HD_720P
  FULL_HD_1080P
  QHD_1440P
  UHD_4K
}

enum TagType {
  GENRE
  COUNTRY
  RATING
  FEATURE
}

enum Suitability {
  PRIME_TIME
  STANDARD
  RESTRICTED
}

enum ReactionType {
  HEART
  FIRE
  CLAP
  WOW
  SAD
  ANGRY
}

enum NotificationType {
  NEW_FOLLOWER
  NEW_VIDEO
  LIVE_START
  LIVE_ENDED
  COMMENT_REPLY
  COMMENT_MENTION
  REACTION
```

```
  SYSTEM_ANNOUNCEMENT
}

enum WatchSource {
  CHANNEL_PAGE
  SEARCH
  RECOMMENDATION
  DIRECT_LINK
  SOCIAL_SHARE
}

// ═══════════════════════════════════════════════════════════
// MODULE 1: AUTH & USER MANAGEMENT
// ═══════════════════════════════════════════════════════════

model User {
  id              String    @id @default(uuid())
  email           String    @unique
  passwordHash    String
  fullName        String
  avatarUrl       String?
  bio             String?   @db.VarChar(500)
  role            UserRole  @default(VIEWER)
  isActive        Boolean   @default(true)
  emailVerified   Boolean   @default(false)
  lastLoginAt     DateTime?
  createdAt       DateTime  @default(now())
  updatedAt       DateTime  @updatedAt

  refreshTokens    RefreshToken[]
  watchlist        Watchlist[]
  notifications    Notification[]
  followedChannels Follow[]
  comments         Comment[]
  reactions        Reaction[]
  watchHistory     WatchHistory[]
  ownedChannels    LiveChannel[]

  @@index([email])
  @@index([role])
}

model RefreshToken {
  id        String   @id @default(uuid())
  token     String   @unique
  userId    String
  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  expiresAt DateTime
  createdAt DateTime @default(now())
  isRevoked Boolean  @default(false)

  @@index([userId])
  @@index([expiresAt])
}

// ═══════════════════════════════════════════════════════════
// MODULE 2: EPG (BROADCAST TV)
// ═══════════════════════════════════════════════════════════

model BroadcastChannel {
  id            String   @id @default(uuid())
  channelCode   String   @unique
  name          String
  shortName     String?
  logoUrl       String?
```

```
  description   String?   @db.VarChar(1000)
  category      BroadcastCategory
  region        String?
  isActive      Boolean   @default(true)
  sortOrder     Int       @default(0)
  createdAt     DateTime  @default(now())
  updatedAt     DateTime  @updatedAt

  programs      BroadcastProgram[]

  @@index([channelCode])
  @@index([category])
  @@index([isActive, sortOrder])
}

model BroadcastProgram {
  id              String    @id @default(uuid())
  title           String
  description     String?   @db.VarChar(2000)
  genre           String?

  airDate         DateTime @db.Date
  startTime       DateTime
  endTime         DateTime
  durationMinutes Int
  isRepeat        Boolean   @default(false)

  thumbnailUrl    String?
  trailerUrl      String?
  trailerSource   ExternalPlatform?

  channelId      String
  channel        BroadcastChannel @relation(fields: [channelId], references: [id])

  tags           ProductionTag[]
  aiReport       BroadcastAiReport?
  isLiveNow      Boolean   @default(false)
  createdAt      DateTime  @default(now())
  updatedAt      DateTime  @updatedAt

  @@unique([channelId, airDate, startTime])
  @@index([airDate])
  @@index([channelId, airDate])
  @@index([isLiveNow])
}

model ProductionTag {
  id       String  @id @default(uuid())
  name     String  @unique
  slug     String  @unique
  color    String?
  type     TagType

  programs BroadcastProgram[]

  @@index([type])
}

model BroadcastAiReport {
  id               String   @id @default(uuid())
  programId        String   @unique
  program          BroadcastProgram @relation(fields: [programId], references: [id], onDelete: Cascade)

  suitability      Suitability
  suitabilityScore  Float
```

```
  sentiment         String
  sentimentScore    Float

  suggestedTimes    String[]
  targetAudience    String[]
  contentWarnings   String[]
  keywords          String[]

  sentimentAgentId  String?
  complianceAgentId String?
  editorAgentId     String?

  generatedAt       DateTime @default(now())

  @@index([suitability])
}

// ═══════════════════════════════════════════════════════════
// MODULE 3: LIVE STREAMING & VOD
// ═══════════════════════════════════════════════════════════

model LiveChannel {
  id            String   @id @default(uuid())

  name          String
  slug          String   @unique
  description   String?   @db.VarChar(1500)

  avatarUrl     String?
  coverUrl      String?
  bannerColor   String?
  tagline       String?   @db.VarChar(200)

  category      LiveCategory
  subcategory   String?
  language      String    @default("vi")
  region        String?

  followerCount    Int       @default(0)
  totalViews       BigInt    @default(0)
  totalVideos      Int       @default(0)
  subscriberCount   Int       @default(0)

  isVerified    Boolean   @default(false)
  isActive      Boolean   @default(true)
  isFeatured    Boolean   @default(false)

  ownerId       String
  owner         User      @relation(fields: [ownerId], references: [id])

  isPublic      Boolean   @default(true)
  allowComments Boolean   @default(true)
  requireSub    Boolean   @default(false)

  createdAt     DateTime  @default(now())
  updatedAt     DateTime  @updatedAt

  liveEvents    LiveEvent[]
  recordings    Recording[]
  followers     Follow[]

  @@index([category])
  @@index([slug])
  @@index([isFeatured, followerCount])
  @@index([ownerId])
```

```
}

model LiveEvent {
  id            String   @id @default(uuid())

  title         String
  description   String?   @db.VarChar(3000)
  thumbnailUrl  String?

  // Stream Configuration
  streamSource       ContentSource @default(EXTERNAL)
  externalPlatform   ExternalPlatform?
  externalId         String?
  externalUrl        String?
  embedCode          String?

  // Internal Stream
  streamUrl          String?
  streamKey          String?
  isPrivate          Boolean   @default(false)
  streamPassword     String?

  quality        StreamQuality @default(AUTO)
  language       String    @default("vi")

  status         EventStatus @default(SCHEDULED)

  scheduledAt    DateTime
  startedAt      DateTime?
  endedAt        DateTime?
  duration       Int?

  viewerCount    Int       @default(0)
  peakViewers    Int       @default(0)
  likeCount      Int       @default(0)
  commentCount   Int       @default(0)
  shareCount     Int       @default(0)

  channelId      String
  channel        LiveChannel @relation(fields: [channelId], references: [id], onDelete: Cascade)

  tags           String[]

  autoRecord     Boolean   @default(true)
  recording      Recording?

  slowMode       Boolean   @default(false)
  chatEnabled    Boolean   @default(true)

  comments       Comment[]
  reactions      Reaction[]

  createdAt      DateTime  @default(now())
  updatedAt      DateTime  @updatedAt

  @@index([status, scheduledAt])
  @@index([channelId, status])
  @@index([status, startedAt])
}

model Recording {
  id            String   @id @default(uuid())

  title         String
  description   String?   @db.VarChar(3000)
```

```
  thumbnailUrl  String?

  // External Source
  contentSource       ContentSource @default(EXTERNAL)
  externalPlatform    ExternalPlatform?
  externalId          String?
  externalUrl         String?
  embedCode           String?

  // Internal Upload
  videoUrl            String?
  videoBucketPath     String?
  fileSize            BigInt?

  // Generated from Live
  generatedFromEventId String?   @unique
  generatedFromEvent   LiveEvent? @relation(fields: [generatedFromEventId], references: [id])

  duration          Int
  quality           StreamQuality
  language          String    @default("vi")

  // Audio
  audioUrl          String?
  audioBucketPath   String?

  viewCount         BigInt    @default(0)
  likeCount         Int       @default(0)
  commentCount      Int       @default(0)
  shareCount        Int       @default(0)
  downloadCount     Int       @default(0)

  contentType       ContentType @default(VIDEO)

  channelId         String
  channel           LiveChannel @relation(fields: [channelId], references: [id], onDelete: Cascade)

  isPublished       Boolean   @default(true)
  isFeatured        Boolean   @default(false)
  isAgeRestricted   Boolean   @default(false)

  tags              String[]
  category          LiveCategory?

  chapters          Json?
  subtitles         Json?

  comments          Comment[]
  reactions         Reaction[]
  watchHistory      WatchHistory[]

  createdAt         DateTime  @default(now())
  updatedAt         DateTime  @updatedAt
  publishedAt       DateTime  @default(now())

  @@index([channelId])
  @@index([contentSource])
  @@index([contentType])
  @@index([viewCount])
  @@index([isFeatured, publishedAt])
}

// ═══════════════════════════════════════════════════════════
// MODULE 4: PODCAST & AUDIO
// ═══════════════════════════════════════════════════════════
```

```
model Podcast {
  id            String   @id @default(uuid())

  title         String
  slug          String   @unique
  description   String?   @db.VarChar(2000)
  coverUrl      String?

  category      LiveCategory @default(PODCAST)
  tags          String[]

  subscriberCount Int     @default(0)
  totalEpisodes   Int     @default(0)
  totalPlays      BigInt  @default(0)

  channelId     String
  channel       LiveChannel @relation(fields: [channelId], references: [id], onDelete: Cascade)

  isExplicit    Boolean   @default(false)
  isComplete    Boolean   @default(false)

  spotifyUrl    String?
  appleUrl      String?
  googleUrl     String?

  episodes      PodcastEpisode[]

  createdAt     DateTime  @default(now())
  updatedAt     DateTime  @updatedAt

  @@index([channelId])
}

model PodcastEpisode {
  id            String   @id @default(uuid())

  title         String
  episodeNumber Int?
  seasonNumber  Int?
  description   String?   @db.VarChar(2000)
  coverUrl      String?

  audioSource         ContentSource @default(EXTERNAL)
  externalPlatform   ExternalPlatform?
  externalUrl        String?

  audioUrl           String?
  audioBucketPath    String?
  fileSize           BigInt?
  duration           Int
  fileFormat         String?   @default("mp3")

  playCount         BigInt    @default(0)
  downloadCount     Int       @default(0)

  podcastId       String
  podcast         Podcast   @relation(fields: [podcastId], references: [id], onDelete: Cascade)

  publishedAt     DateTime
  createdAt       DateTime  @default(now())

  @@index([podcastId, publishedAt])
}
```

```
// ═══════════════════════════════════════════════════════════
// MODULE 5: SOCIAL & ENGAGEMENT
// ═══════════════════════════════════════════════════════════

model Follow {
  id          String   @id @default(uuid())
  followerId  String
  follower    User     @relation(fields: [followerId], references: [id])
  channelId   String
  channel     LiveChannel @relation(fields: [channelId], references: [id], onDelete: Cascade)

  notifyNewVideo   Boolean @default(true)
  notifyLiveStart  Boolean @default(true)
  notifyNewPost    Boolean @default(true)

  createdAt   DateTime @default(now())

  @@unique([followerId, channelId])
  @@index([channelId])
}

model Comment {
  id          String   @id @default(uuid())
  content     String   @db.VarChar(2000)
  userId      String
  user        User     @relation(fields: [userId], references: [id])

  liveEventId String?
  liveEvent   LiveEvent? @relation(fields: [liveEventId], references: [id], onDelete: Cascade)

  recordingId String?
  recording   Recording? @relation(fields: [recordingId], references: [id], onDelete: Cascade)

  parentId    String?
  parent      Comment?  @relation("CommentReplies", fields: [parentId], references: [id])
  replies     Comment[] @relation("CommentReplies")

  likeCount   Int      @default(0)
  isEdited    Boolean  @default(false)
  isPinned    Boolean  @default(false)
  isHidden    Boolean  @default(false)

  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  @@index([liveEventId, createdAt])
  @@index([recordingId, createdAt])
  @@index([parentId])
}

model CommentLike {
  id        String  @id @default(uuid())
  userId    String
  commentId String
  comment   Comment @relation(fields: [commentId], references: [id], onDelete: Cascade)

  createdAt DateTime @default(now())

  @@unique([userId, commentId])
}

model Reaction {
  id          String   @id @default(uuid())
  userId      String
  user        User     @relation(fields: [userId], references: [id])
```

```
  type        ReactionType

  liveEventId String?
  liveEvent   LiveEvent? @relation(fields: [liveEventId], references: [id], onDelete: Cascade)

  recordingId String?
  recording   Recording? @relation(fields: [recordingId], references: [id], onDelete: Cascade)

  createdAt   DateTime @default(now())

  @@unique([userId, liveEventId])
  @@unique([userId, recordingId])
  @@index([liveEventId])
  @@index([recordingId])
}

model WatchHistory {
  id          String   @id @default(uuid())
  userId      String
  user        User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  recordingId String
  recording   Recording @relation(fields: [recordingId], references: [id], onDelete: Cascade)

  progress    Int      @default(0)
  duration    Int
  completed   Boolean  @default(false)
  watchedAt   DateTime @default(now())
  source      WatchSource

  @@unique([userId, recordingId])
  @@index([userId, watchedAt])
}

model Notification {
  id          String   @id @default(uuid())
  userId      String
  user        User     @relation(fields: [userId], references: [id], onDelete: Cascade)

  type        NotificationType
  title       String
  body        String
  imageUrl    String?
  actionUrl   String?

  actorId     String?
  actor       User?    @relation("NotificationActor", fields: [actorId], references: [id])

  eventId     String?
  recordingId String?
  channelId   String?

  isRead      Boolean  @default(false)
  readAt      DateTime?

  createdAt   DateTime @default(now())

  @@index([userId, isRead])
  @@index([userId, createdAt])
}

// ═══════════════════════════════════════════════════════════
// MODULE 6: EPG FEATURES
// ═══════════════════════════════════════════════════════════

model Watchlist {
```

```
  id          String   @id @default(uuid())
  userId      String
  user        User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  programId   String
  program     BroadcastProgram @relation(fields: [programId], references: [id], onDelete: Cascade)

  notifyBefore Boolean @default(true)
  notifyAt    DateTime?

  createdAt   DateTime @default(now())

  @@unique([userId, programId])
}

model ScheduleReminder {
  id          String   @id @default(uuid())
  userId      String
  programId   String
  remindAt    DateTime
  isNotified  Boolean  @default(false)
  notifiedAt  DateTime?
  createdAt   DateTime @default(now())

  @@index([remindAt, isNotified])
}

// ═══════════════════════════════════════════════════════════
// MODULE 7: AUDIT LOG
// ═══════════════════════════════════════════════════════════

model AuditLog {
  id          String   @id @default(uuid())
  userId      String?
  userEmail   String?
  action      String
  entityType  String
  entityId    String?
  oldValues   Json?
  newValues   Json?
  ipAddress   String?
  userAgent   String?

  createdAt   DateTime @default(now())

  @@index([userId, createdAt])
  @@index([entityType, entityId])
  @@index([action, createdAt])
}
```

3. SQL Migrations

3.1 Create Extensions

```
-- Enable UUID extension
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- Enable Full-Text Search
CREATE EXTENSION IF NOT EXISTS "pg_trgm";
```

3.2 Create Tables (Direct SQL for Supabase)

```
-- ═══════════════════════════════════════════════════════════
-- ENUMS (Create as PostgreSQL types)
-- ═══════════════════════════════════════════════════════════

CREATE TYPE "UserRole" AS ENUM ('GUEST', 'VIEWER', 'STAFF', 'ADMIN');
CREATE TYPE "BroadcastCategory" AS ENUM ('GENERAL', 'NEWS', 'ENTERTAINMENT', 'SPORTS', 'MOVIES', 'KIDS',
CREATE TYPE "LiveCategory" AS ENUM ('GAMING', 'PODCAST', 'MUSIC', 'EDUCATION', 'LIFESTYLE', 'NEWS', 'SPOR
CREATE TYPE "ContentSource" AS ENUM ('EXTERNAL', 'UPLOADED', 'GENERATED');
CREATE TYPE "ExternalPlatform" AS ENUM ('YOUTUBE', 'VIMEO', 'TWITCH', 'FACEBOOK', 'TIKTOK', 'DAILYMOTION'
CREATE TYPE "EventStatus" AS ENUM ('SCHEDULED', 'LIVE', 'ENDED', 'CANCELLED', 'ON_DEMAND');
CREATE TYPE "ContentType" AS ENUM ('VIDEO', 'AUDIO', 'LIVE');
CREATE TYPE "StreamQuality" AS ENUM ('AUTO', 'SD_480P', 'HD_720P', 'FULL_HD_1080P', 'QHD_1440P', 'UHD_4K'
CREATE TYPE "TagType" AS ENUM ('GENRE', 'COUNTRY', 'RATING', 'FEATURE');
CREATE TYPE "Suitability" AS ENUM ('PRIME_TIME', 'STANDARD', 'RESTRICTED');
CREATE TYPE "ReactionType" AS ENUM ('HEART', 'FIRE', 'CLAP', 'WOW', 'SAD', 'ANGRY');
CREATE TYPE "NotificationType" AS ENUM ('NEW_FOLLOWER', 'NEW_VIDEO', 'LIVE_START', 'LIVE_ENDED', 'COMMENT
CREATE TYPE "WatchSource" AS ENUM ('CHANNEL_PAGE', 'SEARCH', 'RECOMMENDATION', 'DIRECT_LINK', 'SOCIAL_SHA

-- ═══════════════════════════════════════════════════════════
-- CORE TABLES
-- ═══════════════════════════════════════════════════════════

-- Users
CREATE TABLE "User" (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  email VARCHAR(255) UNIQUE NOT NULL,
  "passwordHash" VARCHAR(255) NOT NULL,
  "fullName" VARCHAR(255) NOT NULL,
  "avatarUrl" VARCHAR(500),
  bio VARCHAR(500),
  role "UserRole" DEFAULT 'VIEWER',
  "isActive" BOOLEAN DEFAULT true,
  "emailVerified" BOOLEAN DEFAULT false,
  "lastLoginAt" TIMESTAMP,
  "createdAt" TIMESTAMP DEFAULT NOW(),
  "updatedAt" TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_user_email ON "User"(email);
CREATE INDEX idx_user_role ON "User"(role);

-- Refresh Tokens
CREATE TABLE "RefreshToken" (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  token VARCHAR(500) UNIQUE NOT NULL,
  "userId" UUID REFERENCES "User"(id) ON DELETE CASCADE,
  "expiresAt" TIMESTAMP NOT NULL,
  "createdAt" TIMESTAMP DEFAULT NOW(),
  "isRevoked" BOOLEAN DEFAULT false
);
CREATE INDEX idx_refresh_token_user ON "RefreshToken"("userId");
CREATE INDEX idx_refresh_token_expires ON "RefreshToken"("expiresAt");

-- ═══════════════════════════════════════════════════════════
-- MODULE 1: EPG (BROADCAST TV)
-- ═══════════════════════════════════════════════════════════

-- Broadcast Channels
CREATE TABLE "BroadcastChannel" (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  "channelCode" VARCHAR(50) UNIQUE NOT NULL,
  name VARCHAR(255) NOT NULL,
  "shortName" VARCHAR(50),
```

```
  "logoUrl" VARCHAR(500),
  description VARCHAR(1000),
  category "BroadcastCategory" NOT NULL,
  region VARCHAR(100),
  "isActive" BOOLEAN DEFAULT true,
  "sortOrder" INTEGER DEFAULT 0,
  "createdAt" TIMESTAMP DEFAULT NOW(),
  "updatedAt" TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_broadcast_channel_code ON "BroadcastChannel"("channelCode");
CREATE INDEX idx_broadcast_channel_category ON "BroadcastChannel"(category);
CREATE INDEX idx_broadcast_channel_active_sort ON "BroadcastChannel"("isActive", "sortOrder");

-- Production Tags
CREATE TABLE "ProductionTag" (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(100) UNIQUE NOT NULL,
  slug VARCHAR(100) UNIQUE NOT NULL,
  color VARCHAR(20),
  type "TagType" NOT NULL
);
CREATE INDEX idx_production_tag_type ON "ProductionTag"(type);

-- Broadcast Programs
CREATE TABLE "BroadcastProgram" (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  title VARCHAR(255) NOT NULL,
  description VARCHAR(2000),
  genre VARCHAR(100),
  "airDate" DATE NOT NULL,
  "startTime" TIMESTAMP NOT NULL,
  "endTime" TIMESTAMP NOT NULL,
  "durationMinutes" INTEGER NOT NULL,
  "isRepeat" BOOLEAN DEFAULT false,
  "thumbnailUrl" VARCHAR(500),
  "trailerUrl" VARCHAR(500),
  "trailerSource" "ExternalPlatform",
  "channelId" UUID REFERENCES "BroadcastChannel"(id),
  "isLiveNow" BOOLEAN DEFAULT false,
  "createdAt" TIMESTAMP DEFAULT NOW(),
  "updatedAt" TIMESTAMP DEFAULT NOW(),
  UNIQUE("channelId", "airDate", "startTime")
);
CREATE INDEX idx_broadcast_program_air_date ON "BroadcastProgram"("airDate");
CREATE INDEX idx_broadcast_program_channel_date ON "BroadcastProgram"("channelId", "airDate");
CREATE INDEX idx_broadcast_program_live ON "BroadcastProgram"("isLiveNow");

-- Broadcast AI Reports
CREATE TABLE "BroadcastAiReport" (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  "programId" UUID UNIQUE REFERENCES "BroadcastProgram"(id) ON DELETE CASCADE,
  suitability "Suitability" NOT NULL,
  "suitabilityScore" FLOAT,
  sentiment VARCHAR(50),
  "sentimentScore" FLOAT,
  "suggestedTimes" TEXT[],
  "targetAudience" TEXT[],
  "contentWarnings" TEXT[],
  keywords TEXT[],
  "sentimentAgentId" VARCHAR(100),
  "complianceAgentId" VARCHAR(100),
  "editorAgentId" VARCHAR(100),
  "generatedAt" TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_ai_report_suitability ON "BroadcastAiReport"(suitability);
```

```
-- Program-Tag Junction
CREATE TABLE "_BroadcastProgramToProductionTag" (
  "A" UUID REFERENCES "BroadcastProgram"(id) ON DELETE CASCADE,
  "B" UUID REFERENCES "ProductionTag"(id) ON DELETE CASCADE,
  PRIMARY KEY ("A", "B")
);

-- ═══════════════════════════════════════════════════════════
-- MODULE 2: LIVE STREAMING & VOD
-- ═══════════════════════════════════════════════════════════

-- Live Channels (Fictional)
CREATE TABLE "LiveChannel" (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name VARCHAR(255) NOT NULL,
  slug VARCHAR(255) UNIQUE NOT NULL,
  description VARCHAR(1500),
  "avatarUrl" VARCHAR(500),
  "coverUrl" VARCHAR(500),
  "bannerColor" VARCHAR(20),
  tagline VARCHAR(200),
  category "LiveCategory" NOT NULL,
  subcategory VARCHAR(100),
  language VARCHAR(10) DEFAULT 'vi',
  region VARCHAR(100),
  "followerCount" INTEGER DEFAULT 0,
  "totalViews" BIGINT DEFAULT 0,
  "totalVideos" INTEGER DEFAULT 0,
  "subscriberCount" INTEGER DEFAULT 0,
  "isVerified" BOOLEAN DEFAULT false,
  "isActive" BOOLEAN DEFAULT true,
  "isFeatured" BOOLEAN DEFAULT false,
  "ownerId" UUID REFERENCES "User"(id),
  "isPublic" BOOLEAN DEFAULT true,
  "allowComments" BOOLEAN DEFAULT true,
  "requireSub" BOOLEAN DEFAULT false,
  "createdAt" TIMESTAMP DEFAULT NOW(),
  "updatedAt" TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_live_channel_category ON "LiveChannel"(category);
CREATE INDEX idx_live_channel_slug ON "LiveChannel"(slug);
CREATE INDEX idx_live_channel_featured ON "LiveChannel"("isFeatured", "followerCount");
CREATE INDEX idx_live_channel_owner ON "LiveChannel"("ownerId");

-- Live Events
CREATE TABLE "LiveEvent" (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  title VARCHAR(255) NOT NULL,
  description VARCHAR(3000),
  "thumbnailUrl" VARCHAR(500),
  "streamSource" "ContentSource" DEFAULT 'EXTERNAL',
  "externalPlatform" "ExternalPlatform",
  "externalId" VARCHAR(255),
  "externalUrl" VARCHAR(500),
  "embedCode" TEXT,
  "streamUrl" VARCHAR(500),
  "streamKey" VARCHAR(255),
  "isPrivate" BOOLEAN DEFAULT false,
  "streamPassword" VARCHAR(255),
  quality "StreamQuality" DEFAULT 'AUTO',
  language VARCHAR(10) DEFAULT 'vi',
  status "EventStatus" DEFAULT 'SCHEDULED',
  "scheduledAt" TIMESTAMP NOT NULL,
  "startedAt" TIMESTAMP,
```

```
  "endedAt" TIMESTAMP,
  duration INTEGER,
  "viewerCount" INTEGER DEFAULT 0,
  "peakViewers" INTEGER DEFAULT 0,
  "likeCount" INTEGER DEFAULT 0,
  "commentCount" INTEGER DEFAULT 0,
  "shareCount" INTEGER DEFAULT 0,
  "channelId" UUID REFERENCES "LiveChannel"(id) ON DELETE CASCADE,
  tags TEXT[],
  "autoRecord" BOOLEAN DEFAULT true,
  "slowMode" BOOLEAN DEFAULT false,
  "chatEnabled" BOOLEAN DEFAULT true,
  "createdAt" TIMESTAMP DEFAULT NOW(),
  "updatedAt" TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_live_event_status_scheduled ON "LiveEvent"(status, "scheduledAt");
CREATE INDEX idx_live_event_channel_status ON "LiveEvent"("channelId", status);
CREATE INDEX idx_live_event_status_started ON "LiveEvent"(status, "startedAt");

-- Recordings
CREATE TABLE "Recording" (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  title VARCHAR(255) NOT NULL,
  description VARCHAR(3000),
  "thumbnailUrl" VARCHAR(500),
  "contentSource" "ContentSource" DEFAULT 'EXTERNAL',
  "externalPlatform" "ExternalPlatform",
  "externalId" VARCHAR(255),
  "externalUrl" VARCHAR(500),
  "embedCode" TEXT,
  "videoUrl" VARCHAR(500),
  "videoBucketPath" VARCHAR(500),
  "fileSize" BIGINT,
  "generatedFromEventId" UUID UNIQUE,
  duration INTEGER NOT NULL,
  quality "StreamQuality" DEFAULT 'HD_720P',
  language VARCHAR(10) DEFAULT 'vi',
  "audioUrl" VARCHAR(500),
  "audioBucketPath" VARCHAR(500),
  "viewCount" BIGINT DEFAULT 0,
  "likeCount" INTEGER DEFAULT 0,
  "commentCount" INTEGER DEFAULT 0,
  "shareCount" INTEGER DEFAULT 0,
  "downloadCount" INTEGER DEFAULT 0,
  "contentType" "ContentType" DEFAULT 'VIDEO',
  "channelId" UUID REFERENCES "LiveChannel"(id) ON DELETE CASCADE,
  "isPublished" BOOLEAN DEFAULT true,
  "isFeatured" BOOLEAN DEFAULT false,
  "isAgeRestricted" BOOLEAN DEFAULT false,
  tags TEXT[],
  category "LiveCategory",
  chapters JSONB,
  subtitles JSONB,
  "createdAt" TIMESTAMP DEFAULT NOW(),
  "updatedAt" TIMESTAMP DEFAULT NOW(),
  "publishedAt" TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_recording_channel ON "Recording"("channelId");
CREATE INDEX idx_recording_content_source ON "Recording"("contentSource");
CREATE INDEX idx_recording_content_type ON "Recording"("contentType");
CREATE INDEX idx_recording_view_count ON "Recording"("viewCount");
CREATE INDEX idx_recording_featured ON "Recording"("isFeatured", "publishedAt");

-- ═══════════════════════════════════════════════════════════
-- MODULE 3: PODCAST
```

```
-- ═══════════════════════════════════════════════════════════

-- Podcasts
CREATE TABLE "Podcast" (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  title VARCHAR(255) NOT NULL,
  slug VARCHAR(255) UNIQUE NOT NULL,
  description VARCHAR(2000),
  "coverUrl" VARCHAR(500),
  category "LiveCategory" DEFAULT 'PODCAST',
  tags TEXT[],
  "subscriberCount" INTEGER DEFAULT 0,
  "totalEpisodes" INTEGER DEFAULT 0,
  "totalPlays" BIGINT DEFAULT 0,
  "channelId" UUID REFERENCES "LiveChannel"(id) ON DELETE CASCADE,
  "isExplicit" BOOLEAN DEFAULT false,
  "isComplete" BOOLEAN DEFAULT false,
  "spotifyUrl" VARCHAR(500),
  "appleUrl" VARCHAR(500),
  "googleUrl" VARCHAR(500),
  "createdAt" TIMESTAMP DEFAULT NOW(),
  "updatedAt" TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_podcast_channel ON "Podcast"("channelId");

-- Podcast Episodes
CREATE TABLE "PodcastEpisode" (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  title VARCHAR(255) NOT NULL,
  "episodeNumber" INTEGER,
  "seasonNumber" INTEGER,
  description VARCHAR(2000),
  "coverUrl" VARCHAR(500),
  "audioSource" "ContentSource" DEFAULT 'EXTERNAL',
  "externalPlatform" "ExternalPlatform",
  "externalUrl" VARCHAR(500),
  "audioUrl" VARCHAR(500),
  "audioBucketPath" VARCHAR(500),
  "fileSize" BIGINT,
  duration INTEGER NOT NULL,
  "fileFormat" VARCHAR(10) DEFAULT 'mp3',
  "playCount" BIGINT DEFAULT 0,
  "downloadCount" INTEGER DEFAULT 0,
  "podcastId" UUID REFERENCES "Podcast"(id) ON DELETE CASCADE,
  "publishedAt" TIMESTAMP NOT NULL,
  "createdAt" TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_podcast_episode_podcast ON "PodcastEpisode"("podcastId", "publishedAt");

-- ═══════════════════════════════════════════════════════════
-- MODULE 4: SOCIAL & ENGAGEMENT
-- ═══════════════════════════════════════════════════════════

-- Follows
CREATE TABLE "Follow" (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  "followerId" UUID REFERENCES "User"(id),
  "channelId" UUID REFERENCES "LiveChannel"(id) ON DELETE CASCADE,
  "notifyNewVideo" BOOLEAN DEFAULT true,
  "notifyLiveStart" BOOLEAN DEFAULT true,
  "notifyNewPost" BOOLEAN DEFAULT true,
  "createdAt" TIMESTAMP DEFAULT NOW(),
  UNIQUE("followerId", "channelId")
);
CREATE INDEX idx_follow_channel ON "Follow"("channelId");
```

```
-- Comments
CREATE TABLE "Comment" (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  content VARCHAR(2000) NOT NULL,
  "userId" UUID REFERENCES "User"(id),
  "liveEventId" UUID REFERENCES "LiveEvent"(id) ON DELETE CASCADE,
  "recordingId" UUID REFERENCES "Recording"(id) ON DELETE CASCADE,
  "parentId" UUID REFERENCES "Comment"(id) ON DELETE CASCADE,
  "likeCount" INTEGER DEFAULT 0,
  "isEdited" BOOLEAN DEFAULT false,
  "isPinned" BOOLEAN DEFAULT false,
  "isHidden" BOOLEAN DEFAULT false,
  "createdAt" TIMESTAMP DEFAULT NOW(),
  "updatedAt" TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_comment_event ON "Comment"("liveEventId", "createdAt");
CREATE INDEX idx_comment_recording ON "Comment"("recordingId", "createdAt");
CREATE INDEX idx_comment_parent ON "Comment"("parentId");

-- Comment Likes
CREATE TABLE "CommentLike" (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  "userId" UUID,
  "commentId" UUID REFERENCES "Comment"(id) ON DELETE CASCADE,
  "createdAt" TIMESTAMP DEFAULT NOW(),
  UNIQUE("userId", "commentId")
);

-- Reactions
CREATE TABLE "Reaction" (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  "userId" UUID REFERENCES "User"(id),
  type "ReactionType" NOT NULL,
  "liveEventId" UUID REFERENCES "LiveEvent"(id) ON DELETE CASCADE,
  "recordingId" UUID REFERENCES "Recording"(id) ON DELETE CASCADE,
  "createdAt" TIMESTAMP DEFAULT NOW(),
  UNIQUE("userId", "liveEventId"),
  UNIQUE("userId", "recordingId")
);
CREATE INDEX idx_reaction_event ON "Reaction"("liveEventId");
CREATE INDEX idx_reaction_recording ON "Reaction"("recordingId");

-- Watch History
CREATE TABLE "WatchHistory" (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  "userId" UUID REFERENCES "User"(id) ON DELETE CASCADE,
  "recordingId" UUID REFERENCES "Recording"(id) ON DELETE CASCADE,
  progress INTEGER DEFAULT 0,
  duration INTEGER NOT NULL,
  completed BOOLEAN DEFAULT false,
  "watchedAt" TIMESTAMP DEFAULT NOW(),
  source "WatchSource",
  UNIQUE("userId", "recordingId")
);
CREATE INDEX idx_watch_history_user ON "WatchHistory"("userId", "watchedAt");

-- Notifications
CREATE TABLE "Notification" (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  "userId" UUID REFERENCES "User"(id) ON DELETE CASCADE,
  type "NotificationType" NOT NULL,
  title VARCHAR(255) NOT NULL,
  body TEXT NOT NULL,
  "imageUrl" VARCHAR(500),
```

```
  "actionUrl" VARCHAR(500),
  "actorId" UUID REFERENCES "User"(id),
  "eventId" UUID,
  "recordingId" UUID,
  "channelId" UUID,
  "isRead" BOOLEAN DEFAULT false,
  "readAt" TIMESTAMP,
  "createdAt" TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_notification_user_read ON "Notification"("userId", "isRead");
CREATE INDEX idx_notification_user_date ON "Notification"("userId", "createdAt");

-- ═══════════════════════════════════════════════════════════
-- MODULE 5: EPG FEATURES
-- ═══════════════════════════════════════════════════════════

-- Watchlist
CREATE TABLE "Watchlist" (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  "userId" UUID REFERENCES "User"(id) ON DELETE CASCADE,
  "programId" UUID REFERENCES "BroadcastProgram"(id) ON DELETE CASCADE,
  "notifyBefore" BOOLEAN DEFAULT true,
  "notifyAt" TIMESTAMP,
  "createdAt" TIMESTAMP DEFAULT NOW(),
  UNIQUE("userId", "programId")
);

-- Schedule Reminders
CREATE TABLE "ScheduleReminder" (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  "userId" VARCHAR(255) NOT NULL,
  "programId" VARCHAR(255) NOT NULL,
  "remindAt" TIMESTAMP NOT NULL,
  "isNotified" BOOLEAN DEFAULT false,
  "notifiedAt" TIMESTAMP,
  "createdAt" TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_reminder_notify ON "ScheduleReminder"("remindAt", "isNotified");

-- ═══════════════════════════════════════════════════════════
-- MODULE 6: AUDIT LOG
-- ═══════════════════════════════════════════════════════════

CREATE TABLE "AuditLog" (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  "userId" UUID,
  "userEmail" VARCHAR(255),
  action VARCHAR(100) NOT NULL,
  "entityType" VARCHAR(100),
  "entityId" UUID,
  "oldValues" JSONB,
  "newValues" JSONB,
  "ipAddress" VARCHAR(50),
  "userAgent" TEXT,
  "createdAt" TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_audit_user ON "AuditLog"("userId", "createdAt");
CREATE INDEX idx_audit_entity ON "AuditLog"("entityType", "entityId");
CREATE INDEX idx_audit_action ON "AuditLog"(action, "createdAt");
```

4. Entity Relationship Diagram

```
erDiagram
    User ||--o{ RefreshToken : has
    User ||--o{ Watchlist : has
    User ||--o{ Follow : follows
    User ||--o{ Comment : writes
    User ||--o{ Reaction : reacts
    User ||--o{ WatchHistory : has
    User ||--o{ Notification : receives
    User ||--o{ LiveChannel : owns

    LiveChannel ||--o{ Follow : has
    LiveChannel ||--o{ LiveEvent : hosts
    LiveChannel ||--o{ Recording : publishes
    LiveChannel ||--o{ Podcast : runs

    LiveEvent ||--o| Recording : generates
    LiveEvent ||--o{ Comment : has
    LiveEvent ||--o{ Reaction : has

    Recording ||--o{ Comment : has
    Recording ||--o{ Reaction : has
    Recording ||--o{ WatchHistory : has

    Podcast ||--o{ PodcastEpisode : contains

    Comment ||o--o{ Comment : replies_to
    Comment ||--o{ CommentLike : has

    BroadcastChannel ||--o{ BroadcastProgram : broadcasts
    BroadcastProgram ||--o| BroadcastAiReport : has
    BroadcastProgram ||--o{ ProductionTag : tagged
    BroadcastProgram ||--o{ Watchlist : saved_in
```

5. Sample Data

5.1 Live Channels (10 Fictional Channels)


```
[
  {
    "name": "OmniGaming",
    "slug": "omni-gaming",
    "category": "GAMING",
    "description": "Kênh gaming hàng đầu Việt Nam - Stream game, giải đấu esports, review game mới nhất",
    "followerCount": 125000,
    "totalViews": 15000000,
    "totalVideos": 450,
    "tags": ["Gaming", "Esports", "Live Stream", "Việt Nam"],
    "avatarUrl": "https://placeholder.co/200×200/6366f1/ffffff?text=OG",
    "bannerColor": "#6366f1"
  },
  {
    "name": "TechTalk Vietnam",
    "slug": "techtalk-vietnam",
    "category": "PODCAST",
    "description": "Podcast công nghệ hàng đầu Việt Nam - Tin tức tech, review gadget, interview founder
    "followerCount": 89000,
    "totalViews": 5200000,
    "totalVideos": 156,
    "tags": ["Công nghệ", "Podcast", "Tech News", "Vietnam"],
    "avatarUrl": "https://placeholder.co/200×200/10b981/ffffff?text=TT",
    "bannerColor": "#10b981"
  },
  {
    "name": "MusicHub",
    "slug": "music-hub",
    "category": "MUSIC",
    "description": "Nhạc Việt, US-UK, K-Pop - Live concert, MV premiere, music news, album launch",
    "followerCount": 200000,
    "totalViews": 25000000,
    "totalVideos": 320,
    "tags": ["Music", "Concert", "Live", "K-Pop", "V-Pop"],
    "avatarUrl": "https://placeholder.co/200×200/ec4899/ffffff?text=MH",
    "bannerColor": "#ec4899"
  },
  {
    "name": "EduStream",
    "slug": "edu-stream",
    "category": "EDUCATION",
    "description": "Học tập trực tuyến - Luyện thi đại học, kỹ năng mềm, lập trình, ngoại ngữ",
    "followerCount": 65000,
    "totalViews": 3800000,
    "totalVideos": 280,
    "tags": ["Education", "Học tập", "Luyện thi", "Programming"],
    "avatarUrl": "https://placeholder.co/200×200/f59e0b/ffffff?text=ES",
    "bannerColor": "#f59e0b"
  },
  {
    "name": "LifestyleVN",
    "slug": "lifestyle-vn",
    "category": "LIFESTYLE",
    "description": "Ẩm thực, du lịch, phong cách sống - Review nhà hàng, travel vlog, tips hay cuộc sống",
    "followerCount": 95000,
    "totalViews": 7800000,
    "totalVideos": 180,
    "tags": ["Lifestyle", "Food", "Travel", "Vietnam"],
    "avatarUrl": "https://placeholder.co/200×200/8b5cf6/ffffff?text=LS",
```

```
    "bannerColor": "#8b5cf6"
  },
  {
    "name": "OmniNews",
    "slug": "omni-news",
    "category": "NEWS",
    "description": "Tin tức 24/7 - Thời sự, kinh tế, thể thao, giải trí, thời tiết",
    "followerCount": 180000,
    "totalViews": 42000000,
    "totalVideos": 1200,
    "tags": ["News", "Tin tức", "24/7"],
    "avatarUrl": "https://placeholder.co/200×200/ef4444/ffffff?text=ON",
    "bannerColor": "#ef4444"
  },
  {
    "name": "SportsArena",
    "slug": "sports-arena",
    "category": "SPORTS",
    "description": "Thể thao - Bóng đá, tennis, boxing, eSports tournaments, highlight reels",
    "followerCount": 150000,
    "totalViews": 35000000,
    "totalVideos": 500,
    "tags": ["Sports", "Football", "Tennis", "E-Sports"],
    "avatarUrl": "https://placeholder.co/200×200/22c55e/ffffff?text=SA",
    "bannerColor": "#22c55e"
  },
  {
    "name": "CookWithMe",
    "slug": "cook-with-me",
    "category": "FOOD",
    "description": "Nấu ăn - Recipe, cooking tips, food review, restaurant tours, street food",
    "followerCount": 78000,
    "totalViews": 4500000,
    "totalVideos": 220,
    "tags": ["Food", "Cooking", "Recipe", "Vietnamese Food"],
    "avatarUrl": "https://placeholder.co/200×200/f97316/ffffff?text=CW",
    "bannerColor": "#f97316"
  },
  {
    "name": "TravelDiary",
    "slug": "travel-diary",
    "category": "TRAVEL",
    "description": "Du lịch - Vlog, travel tips, hidden gems, budget travel, backpacking",
    "followerCount": 62000,
    "totalViews": 3200000,
    "totalVideos": 150,
    "tags": ["Travel", "Vlog", "Adventure", "Budget Travel"],
    "avatarUrl": "https://placeholder.co/200×200/06b6d4/ffffff?text=TD",
    "bannerColor": "#06b6d4"
  },
  {
    "name": "ComedyHouse",
    "slug": "comedy-house",
    "category": "ENTERTAINMENT",
    "description": "Hài kịch, stand-up comedy, vlog hài - Giải trí không giới hạn, meme, parody",
    "followerCount": 220000,
    "totalViews": 28000000,
    "totalVideos": 380,
    "tags": ["Comedy", "Stand-up", "Funny", "Vietnamese Humor"],
    "avatarUrl": "https://placeholder.co/200×200/eab308/ffffff?text=CH",
    "bannerColor": "#eab308"
  }
]
```

5.2 Sample Recordings (All Content Sources)

```
[
  {
    "title": "Review iPhone 16 Pro Max - Có đáng lên đời không?",
    "contentSource": "EXTERNAL",
    "externalPlatform": "YOUTUBE",
    "externalId": "dQw4w9WgXcQ",
    "duration": 1200,
    "viewCount": 45000,
    "likeCount": 3200,
    "category": "TECH",
    "tags": ["iPhone", "Apple", "Review", "Smartphone"],
    "contentType": "VIDEO"
  },
  {
    "title": "Highlights: Giải đấu Valorant Championship 2026",
    "contentSource": "GENERATED",
    "duration": 7200,
    "viewCount": 150000,
    "likeCount": 12000,
    "category": "GAMING",
    "tags": ["Valorant", "Championship", "Highlights", "Esports"],
    "contentType": "VIDEO"
  },
  {
    "title": "Workshop React Native cho người mới bắt đầu",
    "contentSource": "UPLOADED",
    "videoUrl": "https://storage.supabase.co/omnicast/workshops/react-native-workshop.mp4",
    "fileSize": 524288000,
    "duration": 10800,
    "viewCount": 25000,
    "likeCount": 1800,
    "category": "EDUCATION",
    "tags": ["React Native", "Workshop", "Programming", "Mobile Dev"],
    "contentType": "VIDEO"
  },
  {
    "title": "TechTalk EP.45: AI trong năm 2026 - Thực tế hay viễn vông?",
    "contentSource": "EXTERNAL",
    "externalPlatform": "YOUTUBE",
    "externalId": "example_video_id_2",
    "duration": 5400,
    "viewCount": 45000,
    "likeCount": 3500,
    "category": "PODCAST",
    "tags": ["AI", "Technology", "Podcast", "TechTalk"],
    "contentType": "AUDIO"
  },
  {
    "title": "Nấu Phở Bò Việt Nam chuẩn vị Hà Nội",
    "contentSource": "UPLOADED",
    "videoUrl": "https://storage.supabase.co/omnicast/recipes/pho-bo.mp4",
    "fileSize": 209715200,
    "duration": 1800,
    "viewCount": 85000,
    "likeCount": 6200,
    "category": "FOOD",
    "tags": ["Phở", "Vietnamese Food", "Recipe", "Cooking"],
    "contentType": "VIDEO"
  }
]
```

5.3 Sample Live Events

```
[
  {
    "title": "Stream rank Đế chế - Rank Thách đấu với viewer",
    "channelSlug": "omni-gaming",
    "status": "SCHEDULED",
    "scheduledAt": "2026-09-25T19:00:00Z",
    "tags": ["Valorant", "Rank", "Stream"],
    "viewerCount": 0,
    "description": "Chúng mình cùng rank Đế chế nào! Viewer vote map, vote agent."
  },
  {
    "title": "Review iPhone 16 Pro Max - Live Review",
    "channelSlug": "techtalk-vietnam",
    "status": "LIVE",
    "startedAt": "2026-09-23T10:00:00Z",
    "tags": ["iPhone", "Apple", "Review", "Live"],
    "viewerCount": 2450,
    "peakViewers": 3200,
    "description": "Unbox và review chi tiết iPhone 16 Pro Max - So sánh với 15 Pro"
  },
  {
    "title": "Live Concert: Summer Vibes 2026",
    "channelSlug": "music-hub",
    "status": "ENDED",
    "startedAt": "2026-09-20T20:00:00Z",
    "endedAt": "2026-09-20T23:30:00Z",
    "tags": ["Concert", "Live Music", "Summer"],
    "peakViewers": 45000,
    "description": "Live concert mùa hè với các nghệ sĩ hot nhất"
  }
]
```

6. API Endpoints

6.1 Live Channels

Method	Endpoint	Description
GET	`/api/live/channels`	List all channels (filter by category)
GET	`/api/live/channels/:slug`	Get channel by slug
POST	`/api/live/channels`	Create channel (Staff only)
PUT	`/api/live/channels/:id`	Update channel
DELETE	`/api/live/channels/:id`	Delete channel

6.2 Live Events

Method	Endpoint	Description
GET	`/api/live/events`	List events (filter: status, date, channel)
GET	`/api/live/events/live-now`	Get currently live events

Method	Endpoint	Description
GET	`/api/live/events/upcoming`	Get upcoming events
GET	`/api/live/events/:id`	Get event details
POST	`/api/live/events`	Create event
PUT	`/api/live/events/:id`	Update event
POST	`/api/live/events/:id/start`	Start live streaming
POST	`/api/live/events/:id/end`	End live streaming

6.3 Recordings

Method	Endpoint	Description
GET	`/api/live/recordings`	List recordings
GET	`/api/live/recordings/:id`	Get recording details
POST	`/api/live/recordings`	Create recording
PUT	`/api/live/recordings/:id`	Update recording

6.4 Social

Method	Endpoint	Description
POST	`/api/live/channels/:id/follow`	Follow channel
DELETE	`/api/live/channels/:id/follow`	Unfollow channel
GET	`/api/live/channels/:id/followers`	Get followers
GET	`/api/live/events/:id/comments`	Get comments
POST	`/api/live/events/:id/comments`	Add comment
POST	`/api/live/events/:id/reactions`	Add reaction
GET	`/api/live/recordings/:id/reactions`	Get reactions

Deployment Instructions

Option 1: Using Prisma (Recommended)

```
# 1. Install dependencies
npm install @prisma/client prisma

# 2. Create .env file
echo 'DATABASE_URL="postgresql://postgres:[YOUR-PASSWORD]@db.tktexqtqfnlfynbytjpw.supabase.co:5432/postgr

# 3. Create prisma folder
mkdir -p prisma

# 4. Copy schema.prisma content to prisma/schema.prisma

# 5. Generate Prisma Client
npx prisma generate

# 6. Push schema to database
npx prisma db push
```

Option 2: Direct SQL (Supabase Dashboard)

1. Go to Supabase Dashboard
2. Navigate to SQL Editor
3. Copy and paste the SQL from Section 3.2
4. Click Run

Option 3: Migration Files

```
# Create migration
npx prisma migrate dev --name init

# Or reset
npx prisma migrate reset
```

Environment Variables

```
# .env
DATABASE_URL="postgresql://postgres:[YOUR-PASSWORD]@db.tktexqtqfnlfynbytjpw.supabase.co:5432/postgres"
NEXT_PUBLIC_SUPABASE_URL="https://tktexqtqfnlfynbytjpw.supabase.co"
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY="sb_publishable_sZa2lJxnHmCn4DsesvFl2g_pU6Q7FQy"
```

Document Version: 2.0
Last Updated: 2026-09-23